PYTHON PROGRAMMING / WEEK 12 / SESSION 2

입력 검증과 기능 테스트·코드 리뷰

MVP의 신뢰성을 유지하면서 기능 하나 확장하기

120분, 휴식 포함 / 담당: MinGi KYUNG

한국어판

오늘의 결과물

- 기능 하나와 그 동작을 확인하는 테스트를 완성한다.
- 실패해도 기존 보고서가 보존되는지 확인한다.
- 개인 저장소 또는 예제 복사본에서 작업한다.

출발점: 기존 테스트

11주차 MVP의 기존 테스트 12개가 통과하는지 먼저 확인한다.

```
cd 11week/examples/korea_market_mvp
python3 -m unittest -v
```

활동 A: 실행 전 판정

활동 A 전체 5분. 먼저 혼자 판정한다.

함수에 직접 전달할 값	허용 / 거절?
0, 100, -1	이유도 적기
True, "100", 1.5	이유도 적기

활동 A: 실행과 비교

저장소 루트에서 실행한다.

```
python3 12week/examples/validation_lab/playground.py verdict
```

- 짝과 판정이 달랐던 값 하나를 고른다.
- 함수 계약으로 결론을 설명한다.

CSV가 들어오는 경계

- CSV 셀은 문자열로 들어온다.
- 열 구조와 필수 필드를 먼저 검사한다.
- 검증·변환한 값을 분석 함수로 전달한다.

종목 코드는 문자열

```
int("005930")  # 5930
```

- 앞자리 0은 종목 코드의 일부다.
- 계산에 쓰는 가격·거래량만 정수로 변환한다.

변환 성공과 유효한 값

```
int("-1")  # -1
```

- 변환에 성공해도 음수 거래량은 거절한다.
- 가격은 0보다 커야 하고 거래량은 0 이상이어야 한다.

행 규칙과 전체 규칙

검사 범위	규칙
행 하나	실제 날짜, 가격·거래량 범위
전체 입력	날짜 하나, 중복 없는 종목 코드

공백 정리 여부도 입력 계약에 포함한다.

최소 거래량의 자료형 계약

- 0 이상의 int만 허용한다.
- bool·문자열·실수는 거절한다.
- 함수 직접 호출과 CLI의 문자열 변환을 구분한다.

입력을 거절하는 코드

```
def validate_min_volume(value):
    if type(value) is not int or value < 0:
        raise ValueError("min_volume must be a nonnegative integer")
    return value
```

isinstance(True, int)는 True이므로 bool을 별도로 고려한다.

예외를 발생시키는 위치

- 계산 규칙은 계약 위반에 ValueError를 발생시킨다.
- 저장 모듈은 파일·디코딩 실패를 호출자에게 전달한다.

예외를 처리하는 위치

- CLI는 예상한 실패를 사용자 메시지와 종료 코드로 바꾼다.
- 의미 있는 대응을 할 수 있는 위치에서 처리한다.
- 예상하지 못한 코드 오류는 개발 중 드러나게 둔다.

실패를 빈 결과로 바꾸는 코드

```
try:
    quotes = load_quotes(path)
except Exception:
    quotes = []
```

파일 누락과 코드 오류가 모두 정상 빈 입력처럼 보인다.

실패와 정상 빈 결과

입력	기대 동작
누락된 CSV	실패, 기존 보고서 보존
헤더만 있는 정상 CSV	성공, 빈 보고서 생성

완전히 빈 파일은 헤더만 있는 파일과 다르다.

기존 CLI의 처리 코드

```
except (OSError, ValueError, UnicodeError) as error:
    print(f"Report not generated: {error}")
    return 1
```

CSV 행 번호처럼 원인 조사에 필요한 정보를 메시지에 담는다.

종료 코드도 결과다

코드	의미
0	보고서 생성 성공
1	예상한 데이터·파일 실패
2	argparse 인자 오류

메시지뿐 아니라 출력 파일 상태도 확인한다.

활동 B: CSV 결과 예측

활동 B 전체 10분. 실행 전에 두 가지를 예측한다.

- 정상 / 헤더만 있음 / 파일 누락 / 음수 거래량.
- 각 경우의 반환 코드와 OLD 보고서 보존 여부를 적는다.

활동 B: 실행과 사례 추가

```
python3 12week/examples/validation_lab/playground.py csv
```

- cases에 잘못된 날짜 또는 필수 열 누락 사례를 추가한다.
- 한 번에 한 조건만 바꾸고 예상·실제를 비교한다.

활동 B: 관찰 설명

- 정상 빈 결과는 왜 OLD를 교체할까?
- 입력 오류는 왜 OLD를 보존해야 할까?
- 활동지에 실행 증거와 이유를 각각 한 문장으로 적는다.

보고서를 교체하기 전

- 모든 입력을 읽고 검증한다.
- 새 HTML 전체를 출력 폴더의 임시 파일에 쓴다.
- 검증·쓰기가 끝날 때까지 기존 보고서를 유지한다.

보고서 교체와 정리

- 쓰기가 끝나면 os.replace로 목적 파일을 교체한다.
- 교체에 실패하면 임시 파일을 정리한다.

범위: 처리 가능한 교체 전 실패에서 기존 결과를 보호한다. 전원 차단 내구성까지 보장하지 않는다.

단위 테스트: 계산 규칙

```
self.assertAlmostEqual(change_pct(100, 110), 10.0)
```

- 작은 함수 하나의 계약을 확인한다.
- 실패하면 조사할 범위가 좁다.

통합 테스트와 사용자 확인

확인 방법	관찰할 결과
잘못된 CSV로 main 호출	실패 코드, 기존 HTML 보존
보고서를 브라우저에서 열기	시장·라벨·빈 상태 표시

함수 테스트와 실제 사용자 흐름을 함께 확인한다.

임시 폴더로 실험 격리

```
from pathlib import Path
from tempfile import TemporaryDirectory

with TemporaryDirectory() as folder:
    output = Path(folder) / "index.html"
    output.write_text("OLD", encoding="utf-8")
```

제출할 보고서 대신 이 경로로 실패 실험을 진행한다.

드문 저장 실패의 재현

```
with patch("storage.os.replace", side_effect=OSError("disk")):
    write_html(output, "NEW")
```

- 테스트 대상이 참조하는 storage.os.replace를 패치한다.
- CSV 파싱 실험에는 실제 임시 파일을 사용한다.

활동 C: 보고서 구출 예측

활동 C 전체 8분. 교체가 실패할 때의 결과를 먼저 적는다.

- OSError는 호출자에게 전달될까?
- 출력은 OLD와 NEW 중 무엇일까?
- 폴더에 임시 파일이 남을까?

활동 C: 관찰을 assertion으로

```
python3 12week/examples/validation_lab/playground.py rescue
```

- assertRaises로 예외 전달을 검사한다.
- assertEqual로 OLD 보존과 남은 파일 목록을 검사한다.
- 활동지의 test_rescue.py 뼈대를 완성하고 실행한다.

새 기능: 최소 거래량

사용자 목표: 일정 거래량 이상인 종목만 비교한다.

```
analyze(quotes, market, min_volume=0)
```

기본값 0이면 기존 두 인자 호출과 결과를 유지한다.

필터 전 검증과 필터 후 정렬

- 중복 코드·혼합 날짜는 전체 입력에서 먼저 거절한다.
- 시장 조건과 최소 거래량 조건을 함께 적용한다.
- 등락률 내림차순·코드 오름차순과 원본 보존을 유지한다.

수락 사례: 기준과 같은 값

거래량	기준 100에서
99	제외
100	포함
101	포함

기대 결과는 구현 코드가 아니라 계약에서 정한다.

수락 사례: 오류와 빈 결과

입력	기대 결과
-1, True, "100", 1.5	각각 ValueError
모든 종목 제외	[], 요약 count=0, mean=None
필터로 숨겨질 중복	ValueError

활동 D: 결함 후보와 최소 입력

활동 D 전체 8분. 각 후보를 잡는 최소 입력을 만든다.

- A: volume > threshold로 비교한다.
- B: isinstance(value, int)로만 자료형을 검사한다.
- C: 필터링한 뒤 중복 코드를 검사한다.

활동 D: 입력 교환과 테스트 계획

- 짝에게 후보 이름을 가리고 입력과 기대 결과를 보여준다.
- 짝은 어떤 결함을 잡는 입력인지 설명한다.
- 경계값·숨겨진 중복 사례를 프로젝트 테스트 계획에 옮긴다.

휴식 — 10분

개인 프로젝트에서 확장할 기능 하나를 정한다.

후반에는 계약과 실패 테스트부터 작성한다.

기능 계약 작성 — 10분 중 앞부분

- labs/session2/project-review.md를 연다.
- 요구사항 ID와 변경할 함수를 연결한다.
- 허용 입력과 정상·오류 결과를 짧게 적는다.

구현 전 실패 증거

- 새 동작을 확인하는 테스트를 먼저 추가한다.
- 기존 코드에서 실행해 예상한 이유로 실패하는지 확인한다.
- 입력·기대 결과·실제 실패를 기록한다.

구현 — 20분 블록

- 새 매개변수를 검증한 뒤 필터를 적용한다.
- 전체 스냅샷 검증을 필터 앞으로 유지한다.
- 원본 입력을 변경하지 않는다.

새 테스트와 회귀 테스트

- 새 사례와 기존 테스트 전체를 실행한다.
- 기본값 0과 기존 두 인자 호출을 확인한다.
- 실패하면 입력을 재현하고 원인을 좁힌다.

결과 시연 준비 — 10분 블록

- 기존 main 진입점으로 정상·빈 보고서를 생성한다.
- 새 함수 동작을 보여줄 테스트를 준비한다.
- CLI 연결이 없으면 보고서와 함수 테스트를 각각 시연한다.

활동 E: 90초 사용자 시연

- 30초: 사용자가 정상 보고서의 시장·종목 수·순위를 설명한다.
- 30초: 빈 보고서의 안내가 충분한지 말한다.
- 30초: 구현자가 기준과 같은 값을 포함하는 테스트를 실행한다.

시연 관찰 기록

- 관찰한 문제 하나 또는 확인한 수락 기준 하나를 적는다.
- 결과를 project-review.md의 실제 시연 항목에 연결한다.
- 최종 제출 테스트는 모두 통과시킨다.

코드 리뷰: 반례 찾기 — 15분 블록

- diff를 계약과 테스트 계획에 대조한다.
- 경계값·오류·빈 결과에서 반례 하나를 찾는다.
- 동료가 없으면 같은 질문으로 자기 diff를 검토한다.

코드 리뷰: 구체적인 의견

```
입력: ______
기대 결과 / 실제 결과: ______ / ______
근거 함수 또는 테스트: ______
```

예외 전달·기존 결과 보존과 이름·책임 분리도 확인한다.

리뷰 반영과 재확인

- 의견에 따라 수정하고 관련 테스트를 다시 실행한다.
- 반영하지 않은 의견은 이유를 기록한다.
- 리뷰 의견과 변경 결과를 제출 양식에 남긴다.

PR에 남길 증거

- 문제와 사용자에게 보이는 변화, 요구사항 ID.
- 변경 함수, 정확한 검증 명령과 실제 결과.
- GitHub 사용이 어려우면 로컬 diff와 리뷰 기록을 제출한다.

제출과 퇴실 질문 — 5분

- 기능 코드·테스트·리뷰 기록과 추적 문서를 제출한다.
- 내가 쓴 테스트가 잡는 구체적인 결함은 무엇인가?
- 필터로 숨겨질 중복 코드도 거절해야 하는 이유는?

13주차로 이어가기

- 기존 MVP가 계속 실행되는지 마지막으로 확인한다.
- 다음 수업에서는 검증된 규칙 위에 데이터 처리나 UI를 확장한다.